6.20 控制器——微程序控制器（设计流程）

考纲定位：计算机组成原理 · 第 6 章 中央处理器（6.20） | 重要度 ★★★ | 常见考法：选择题（设计步骤、硬布线 vs 微程序对比）

一句话核心：微程序控制器设计 = 分析数据通路 → 划分微操作 → 设计微指令格式 → 编写微程序 → 构建地址转移逻辑

微程序控制器的设计流程

① 对指令集中每一条指令执行时的数据通路进行分析，得出每一条指令的执行操作流程和每一步操作所需的控制信号 。

② 将每一条指令的每一个微操作划分到相应状态周期的具体节拍信号上，也就是对操作控制信号进行定时控制。

③ 以时钟周期为单位画出指令执行的状态转换图，得出状态转换表，构造相应的有限状态机FSM。

④ 设计微指令格式（水平型微指令和垂直型微指令）、微命令编码方法（操作控制的直接表示法、编码表示法以及混合表示法）。

⑤ 根据指令执行的有限状态机FSM编写指令集中每一条指令相应的微程序，按照FSM将全部微程序排列并存放到控制存储器CS中。

⑥ 根据全部微程序在CS中的组织方式，构建微程序控制器中的地址转移逻辑、微地址寄存器μAR、控制存储器CS之间的通路，进而实现微程序控制器。

本图定位：从指令到微程序控制器的完整设计流程

考点/易错

- 先分析每条指令的数据通路和执行流程，得到微操作序列
- 把微操作划分到具体节拍/时钟周期，确定控制信号时序
- 设计微指令格式（水平/垂直、直接/编码/混合），再编写微程序
- 根据微程序在 CS 中的组织，构建地址转移逻辑、μAR、CS 通路
- 易错：设计流程不是“先写微程序再分析指令”；顺序不能反

补充

- 408 设计题通常按“微操作 → 微指令 → 微程序 → 地址逻辑”给分，步骤要写全
- 微程序控制器设计比硬布线规整，但多了 CS 访问延迟

硬布线 vs 微程序

微程序控制器与硬布线控制器的对比

对比项目	硬布线控制器	微程序控制器
设计思想	微操作控制信号直接由硬件逻辑电路（组合逻辑+时序逻辑）给出	微操作控制信号用微程序实现，微程序存放在控制存储器CS中。现代微程序控制器均采用可写的CS，以便修改、扩展指令功能，甚至修复处理器存在的出厂“bug”。
执行速度	快	慢（需要频繁访问控制存储器CS读取微指令）
设计复杂性	高（尤其是指令集复杂时，电路设计非常复杂）	较低（将复杂性转移到了编写微指令程序）
可修改/扩充	困难	容易
应用场景	RISC处理器（如ARM）	CISC处理器（如x86）

维度	硬布线控制器	微程序控制器
设计思想	组合逻辑直接产生控制信号	微程序存储在 CS，取微指令产生控制信号
执行速度	快	慢（需访问 CS）
设计复杂度	高，指令复杂时电路庞大	较低，规整
可修改/扩充	困难	容易，改微程序即可
典型应用	RISC（ARM）	CISC（x86）

考点/易错

- 硬布线快但难改；微程序慢但灵活
- 现代 CPU 常混合：简单指令硬布线，复杂指令微程序
- 易错：微程序控制器不是“软件实现”，它仍是硬件读取微指令；只是控制逻辑以存储形式存在

补充

- 408 常问“RISC 为什么多用硬布线”：指令规整、控制简单、追求速度
- CISC 指令复杂，微程序更易维护和扩展

知识关联

- 前置：6.13 控制器——微程序控制器（概述）、6.16 控制器——微程序控制器（微指令的设计）、6.18 控制器——微程序控制器（微指令中操作控制字段的表示方法）、6.19 控制器——微程序控制器（水平和垂直型微指令）
- 对比：6.11 控制器——三级时序的硬布线控制器（概述）、6.12 控制器——现代时序的硬布线控制器
- 全章导航：6.0 导航与术语